AETHER

A Local-First Intelligent Desktop Assistant

Generative AI · Agentic Tool Execution · Desktop Automation

Project	AETHER — Autonomous Desktop Intelligence
Platform	Windows 10/11
Frontend	Electron, React, TypeScript
Backend	Python, FastAPI
Local AI	Ollama + Qwen3 4B (Q4 target)
Target hardware	16 GB RAM · NVIDIA 6 GB VRAM · 256 GB free
AI API dependency	None required for model inference
Submitted by	_____
Roll number	_____
Submitted to	_____
Date	_____

Abstract

AETHER is a local-first desktop assistant designed to interpret natural-language instructions and execute selected productivity actions on a Windows computer. It combines a locally hosted large language model with a controlled tool-execution layer. The prototype supports note creation, reminders, allowlisted application launching, music search, filename search and document generation. A conversational desktop interface displays task progress and results. Language-model inference runs locally through Ollama, avoiding a dependency on paid hosted AI APIs.

1. Problem Statement

Many AI assistants rely on cloud services and are separated from the desktop applications where users perform everyday work. AETHER explores how a local language model can interpret user intent and invoke controlled desktop tools through a single interface, while keeping prompts and personal files on the user's device.

2. Objectives

- Build a natural-language interface for desktop productivity.
- Run LLM inference locally without a paid cloud AI API.
- Route user requests to safe, typed tools.
- Support notes, reminders, app launching, file discovery and document creation.
- Provide a conversational interface with local task history.
- Demonstrate modular design, privacy controls and verifiable execution.

3. Hardware Profile and Model Selection

The target machine has 16 GB system RAM, an NVIDIA GPU with 6 GB VRAM and approximately 256 GB free storage. The default model target is Qwen3 4B in a quantised Ollama distribution. Qwen3 1.7B is the fallback if 4B inference is too slow or memory pressure is high. Actual speed and GPU offloading depend on the exact GPU, CPU, driver, context length and concurrent applications; performance must be measured on the deployment machine.

4. System Architecture

The Electron desktop client communicates with a local FastAPI service. The local agent asks Ollama on localhost to classify a command into structured JSON. The backend validates the selected action and invokes a predefined tool. Task state is stored in SQLite; notes and generated documents are saved in local folders. Local speech-to-text uses faster-whisper. Read-aloud uses the operating system's speech synthesis.

- User enters or dictates a command.
- Electron submits it to the local FastAPI backend.
- The agent requests structured JSON from the local Ollama model.
- The backend validates the action and invokes a predefined tool.
- The system checks the result where possible and updates task state.
- The frontend displays the outcome and execution trace.

5. AI Concepts Demonstrated

Large Language Models: A locally hosted instruction-tuned model interprets requests and generates document content.

Transformer architecture: The selected pretrained model belongs to the Transformer family. Self-attention and multi-head attention operate inside the model; AETHER does not implement or train those layers.

Prompt engineering: A structured system prompt constrains the model to a known action schema and safety rules.

Structured generation and tool routing: The model returns JSON describing an action and arguments; backend code maps the action to a controlled handler.

Agentic workflow: The prototype follows an interpret → act → verify loop. It currently routes one primary action per command rather than robust multi-step planning.

Local inference: Ollama serves the model on localhost, so inference does not require a cloud AI API.

Speech recognition: faster-whisper converts recorded audio into text locally.

Persistence: SQLite stores task history; local folders store notes and generated outputs.

6. Main Modules

Module	Responsibility
Electron / React UI	Chat interface, sidebar, settings, notes and reminders
FastAPI	Local application API and task endpoints
Local agent	Prompt construction, action classification and document generation
Ollama	On-device LLM inference
Tool layer	Notes, reminders, desktop apps, music, files and documents
SQLite	Persistent task history and execution steps
faster-whisper	Local speech transcription

7. Safety and Privacy

- The LLM cannot execute arbitrary shell commands.
- Application launching uses an allowlist.
- File search is read-only.
- Generated files are written to a configured local output directory.
- The local AI runtime is addressed through localhost.
- External websites and music providers still require internet access and their own accounts.
- The first model/dependency downloads require internet; inference can run offline after setup.

8. Current Scope and Limitations

This release is a functional prototype, not a finished commercial assistant. It currently routes one primary action per command. Spotify opens a search page and does not guarantee playback. Reminders are displayed in AETHER but native Windows toast notifications and recurring schedules are not yet implemented. File discovery is filename-based and limited to Desktop, Documents and Downloads. The speech model downloads on first use. Browser automation, semantic RAG, wake-word detection, calendar/email integration, full UI automation and comprehensive permission dialogs are future work. The desktop app has not yet been validated on the target PC.

9. Testing and Evaluation

The repository includes a reproducible functional test plan. Before a final demonstration, install Ollama, download the model, install Python and Node dependencies, launch the backend and Electron client, then execute each test and record the observed result. Do not report tests as passed until they have been run on the target machine.

- Task success rate = completed supported tasks / attempted supported tasks.
- Tool selection accuracy = correctly selected actions / evaluated commands.
- Median end-to-end task latency.
- Speech transcription quality on a fixed sample set.
- Failure recovery rate and false-success rate.

10. Future Enhancements

- Multi-step planner with dependency-aware execution and cancellation.
- RAG over local PDFs and documents using embeddings and FAISS.
- Native Windows notifications and recurring reminders.
- Browser and desktop UI automation with explicit confirmation gates.
- Semantic memory and conversation search.
- Evaluation dashboard for task success, latency and tool selection accuracy.
- Signed installer with runtime bootstrap and model manager.

Conclusion

AETHER demonstrates a practical local-first architecture that combines a pretrained LLM with safe desktop tools. Its contribution is the integration of local language understanding, structured action routing and a desktop productivity interface. The prototype creates a foundation for further work in RAG, agent orchestration, local inference and human-in-the-loop automation.